

React Core Concepts

Components, State, Props

STEP 1: What is React?

React is a JavaScript library (not a full framework) developed by Facebook for building user interfaces, especially for single-page applications.

Its core idea: Break your UI into small, reusable pieces called Components.

STEP 2: Setting Up a React Project (Quick Start)

Refer previous pdf

STEP 3: Your First Component — Function Component

Open App.js in your sandbox. You'll see something like this:

```
import React from 'react';
```

```
function App() {  
  return (  
    <div>  
      <h1>Hello, React!</h1>  
    </div>  
  );  
}
```

```
export default App;
```

Breakdown:

function App()	This is a React component— a JavaScript function that returns JSX
return (...)	Returns JSX (JavaScript XML) — looks like HTML but is actually JavaScript
<div>	JSX element — must have one root element
export default App	Makes this component available to be used elsewhere

STEP 4: What is JSX?

JSX = JavaScript + XML

It lets you write HTML-like syntax inside JavaScript.

```
<h1>Hello World</h1>
```

Under the hood, React converts this into regular JavaScript calls:

```
React.createElement('h1', null, 'Hello World')
```

Important: JSX must be wrapped in a single parent element.

✗ Invalid:

```
return (  
  <h1>Hello</h1>  
  <p>World</p>  
);
```

✓ Valid:

```
return (  
  <div>  
    <h1>Hello</h1>  
    <p>World</p>  
  </div>  
);
```

use Fragments (<>...</>) to avoid extra DOM nodes:

```
return (  
  <>  
    <h1>Hello</h1>  
    <p>World</p>  
  </>  
);
```

STEP 5: Reusable Component — Create Your Own!

Let's make a new component: Button.js

Step 5.1: Create Button.js

In your project, create a new file: Button.js

```
// Button.js
```

```
import React from 'react';
```

```
function Button() {  
  return (  
    <button style={{ padding: '10px', margin: '5px' }}>  
      Click Me!  
    </button>  
  );  
}
```

```
export default Button;
```

Step 5.2: Use it in App.js

```
// App.js
import React from 'react';
import Button from './Button'; // Import your new component
```

```
function App() {
  return (
    <div>
      <h1>Welcome to My App</h1>
      <Button /> {/* Use the Button component */}
      <Button /> {/* Use it again! */}
      <Button /> {/* Reused 3 times! */}
    </div>
  );
}
```

```
export default App;
```

STEP 6: Passing Data with Props

Components become powerful when they accept data.

Example: Make Button Dynamic

Update Button.js:

```
// Button.js
import React from 'react';

function Button({ text, color }) { // Destructure props
  return (
    <button
      style={{
        padding: '10px',
        margin: '5px',
        backgroundColor: color || 'lightblue',
        border: 'none',
        borderRadius: '4px'
      }}
    >
      {text} {/* Render the text passed in */}
    </button>
  );
}
```

```
export default Button;
```

Now update App.js:

```
// App.js
import React from 'react';
import Button from './Button';

function App() {
  return (
```

```

    <div>
      <h1>Dynamic Buttons</h1>
      <Button text="Login" color="blue" />
      <Button text="Logout" color="red" />
      <Button text="Cancel" /> { /* Uses default color */}
    </div>
  );
}

```

```
export default App;
```

Key Concept: Props (Properties)

- props = input to a component.
- Passed as attributes: <Button text="Login" />
- Received inside component as a parameter: function Button(props) or function Button({ text })
- Props are read-only — never modify them inside the component.

STEP 7: State — Making Components Interactive

So far, our buttons just display. What if we want to count clicks?

We use State.

Use useState Hook (Function Component)

Update Button.js:

```

// Button.js
import React, { useState } from 'react'; // Import useState

function Button({ text }) {
  const [count, setCount] = useState(0); // Initialize state

  const handleClick = () => {
    setCount(count + 1); // Update state
  };

  return (
    <button
      onClick={handleClick} // Attach event handler
      style={{ padding: '10px', margin: '5px' }}
    >
      {text} - Clicked {count} times
    </button>
  );
}

```

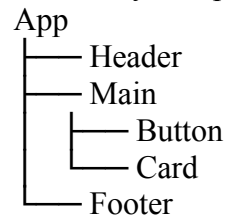
```
export default Button;
```

What happened?

<code>import { useState } from 'react'</code>	Import the state hook
<code>const [count, setCount] = useState(0)</code>	Declare state variable <code>count</code> , initialized to 0. <code>setCount</code> is the function to update the state
<code>onClick={handleClick}</code>	When button is clicked, call <code>handleClick</code>
<code>setCount(count + 1)</code>	Updates state → triggers re-render

STEP 8: Component Hierarchy — Building Complex UIs

Think of your app as a tree:



Each is its own component.

// Header.js

```
import React from 'react';
```

```
function Header() {  
  return <header style={{ padding: '20px', backgroundColor: '#f0f0f0' }}>  
    <h1>My Website</h1>  
  </header>;  
}
```

```
export default Header;
```

Use it in App.js:

// App.js

```
import React from 'react';  
import Header from './Header';  
import Button from './Button';
```

```
function App() {  
  return (  
    <div>  
      <Header />  
      <Button text="Say Hi" />  
    </div>  
  );  
}
```

```
}  
  
export default App;
```

STEP 9: Summary — Core Concepts Recap

Component	A reusable piece of UI (a function that returns JSX)
JSX	Syntax extension that looks like HTML but runs as JavaScript
Props	Inputs to a component (passed from parent to child). Read-only.
State	Internal data that can change over time. Managed with <code>useState</code>
Re-rendering	When state or props change, React updates the UI automatically
Composition	Build complex UIs by combining simple components

STEP 10: Practice Exercise (Do This!)

Build a Counter App with these components:

1. `Counter.js` — Contains state (`count`) and increment/decrement buttons
2. `Display.js` — Shows current count (receives `count` as prop)